INSURE AI

PLATAFORMA INTELIGENTE PARA GESTIÓN DE ACCIDENTES, SINIESTROS Y FORMULARIOS

PLAN MAESTRO DE PRODUCTO, ARQUITECTURA Y DESARROLLO AUTÓNOMO

React · Node.js · Expo · MongoDB · IA generativa · RAG · Computer Vision · BMAD

Versión del documento: 1.0 · Fecha: 7 de noviembre de 2026

Documento de partida para entregar a Claude Code y utilizar como contexto de análisis, planificación, arquitectura, UX, desarrollo y aseguramiento de calidad con BMAD Method.

Estado: propuesta inicial. Las decisiones marcadas como pendientes deben ser confirmadas durante el análisis BMAD antes de convertirse en estándares definitivos.

0. Cómo utilizar este documento

Este PDF es el documento maestro inicial del proyecto. Claude debe leerlo como contexto de producto y BMAD debe convertirlo en artefactos verificables. No es una autorización para generar código sin análisis previo.

1. **Primero:** cargar este PDF en Claude Code como contexto del proyecto y solicitar un diagnóstico de requisitos y vacíos.
2. **Segundo:** ejecutar BMAD Analysis y Planning para producir Product Brief y PRD.
3. **Tercero:** ejecutar Solutioning para validar arquitectura, stack, datos, seguridad, RAG y UX.
4. **Cuarto:** construir el entorno de desarrollo autónomo (Sprint 0), evaluar plantillas y configurar agentes/workflows.
5. **Quinto:** iniciar historias pequeñas con criterios de aceptación, pruebas y Pull Requests.

Importante: “entrenar agentes” en este proyecto significa principalmente configurar su contexto, instrucciones, conocimiento, herramientas, workflows, permisos y evaluaciones. No implica necesariamente entrenar o ajustar los pesos de un modelo fundacional.

1. Visión del producto

Nombre provisional: INSURE AI — Intelligent Insurance & Claims Management Platform.

Construir una super app modular, AI-first y multiplataforma para apoyar la recepción, análisis, clasificación preliminar, documentación y seguimiento de accidentes laborales, personales y de tránsito. El sistema debe analizar relatos, documentos, fotografías y fotogramas extraídos de videos; consultar conocimiento documental mediante RAG; generar formularios y gestionar autorizaciones y flujos de revisión.

Principio rector: la IA asiste y documenta; las decisiones legales, de cobertura, médicas o de autorización definitiva permanecen bajo responsabilidad de personas facultadas.

1.1 Objetivos del piloto

- Registrar usuarios, roles, solicitantes y expedientes.
- Capturar el relato del accidente y recopilar datos faltantes mediante preguntas guiadas.
- Distinguir dimensiones diferentes: tipo de evento (tránsito, caída, etc.) y posible relación laboral/personal.
- Analizar imágenes y frames de video para describir elementos observables, sin afirmar causalidad ni diagnosticar lesiones.
- Consultar pólizas, procedimientos, manuales y normativa autorizada mediante RAG con citas y vigencia.
- Extraer datos estructurados y preparar formularios PDF a partir de plantillas aprobadas.
- Gestionar autorizaciones, revisión humana, estados y auditoría.
- Medir calidad de extracción, recuperación RAG, clasificación y seguridad.

1.2 Fuera del alcance inicial

- Decidir automáticamente si un siniestro está cubierto o rechazar una reclamación.
- Emitir diagnósticos médicos o determinar gravedad clínica a partir de imágenes.
- Reconocimiento facial o identificación biométrica de personas.
- Entrenar un modelo fundacional propio.
- Automatizar pagos, indemnizaciones o decisiones con efectos jurídicos sin aprobación humana.
- Sustituir los sistemas oficiales de aseguradoras, ARL, autoridades o prestadores.

2. Usuarios y canales

Canal	Usuarios principales	Responsabilidades
Web administrativa	Administradores, analistas, revisores	Gestión de expedientes, usuarios, formularios, conocimiento, auditoría y reportes.
Portal web de usuario	Asegurados, trabajadores, terceros	Reportar un evento, aportar evidencias, completar formularios y consultar estado.
App móvil nativa	Usuarios de campo y solicitantes	Captura guiada de fotos/video, cámara, ubicación opcional, carga y seguimiento.
Consola interna IA	Servicios y operadores autorizados	Procesamiento de documentos, RAG, análisis visual, colas y evaluación.

Estrategia multiplataforma: iniciar con una web responsive/PWA en Next.js y diseñar una app nativa posterior con Expo + React Native. Mantener un backend, contratos API, validaciones y reglas de dominio compartidas; no forzar que todas las interfaces sean idénticas.

3. Alcance funcional por módulos

Módulo	Capacidades iniciales
Identidad y acceso	Login, recuperación, sesiones, MFA cuando corresponda, roles, permisos y administración de usuarios.
Expedientes	Crear, consultar, actualizar, asignar, relacionar personas, estados, comentarios y línea de tiempo.
Registro de accidentes	Fecha, hora, lugar, relato, actividad, desplazamiento, empleador, transporte, testigos y preguntas de aclaración.
Evidencias	Carga de imágenes, video y documentos; metadatos, hash, permisos, relación con expediente y estado de procesamiento.
Motor IA	Extracción estructurada, clasificación preliminar, preguntas de seguimiento, incertidumbre y explicación.
RAG / conocimiento	Ingesta, OCR, fragmentación, embeddings, búsqueda híbrida, filtros de vigencia y citas de fuentes.
Formularios	Plantillas versionadas, prellenado, revisión, generación PDF y control de versiones.
Autorizaciones	Solicitudes, aprobadores, delegación controlada, decisión, motivo, firma/aceptación y auditoría.
Administración	Catálogos, políticas, plantillas, usuarios, roles, parámetros y configuración.
Auditoría y calidad IA	Registro de acciones, versiones de prompts/modelos, evaluación, errores y métricas.

4. Arquitectura de referencia

Se recomienda un monorepo modular con límites explícitos entre web, móvil, API, procesamiento IA y paquetes compartidos. Para el piloto puede desplegarse como un backend modular y uno o varios workers; no se recomienda comenzar con una arquitectura de microservicios distribuida sin necesidad demostrada.

Capa	Tecnología candidata	Responsabilidad
Web	Next.js, React, TypeScript	Portal administrativo y portal de usuarios; PWA responsive.
UI	Tailwind CSS, shadcn/ui, Lucide	Sistema visual, accesibilidad y componentes.
Móvil	Expo + React Native	Aplicación Android/iOS nativa cuando el piloto lo justifique.
API	Node.js + NestJS + TypeScript	Autenticación, RBAC, reglas de negocio, expedientes, formularios y auditoría.
Persistencia	MongoDB + Mongoose	Documentos operacionales, usuarios, expedientes, metadatos y auditoría.
Archivos	Object storage compatible S3	Fotos, videos, PDFs y originales; evitar almacenar binarios grandes en MongoDB.

Capa	Tecnología candidata	Responsabilidad
Colas	Redis + BullMQ, si se requiere	OCR, video, embeddings, indexación y notificaciones asíncronas.
IA	Claude API detrás de una capa adaptadora	Razonamiento contextual, extracción y generación estructurada.
RAG	MongoDB Vector Search o Qdrant tras evaluación	Recuperación semántica y/o híbrida con filtros y fuentes.
Video	FFmpeg + servicio de visión multimodal	Extracción controlada de frames y análisis visual.
QA	Vitest/Jest, Playwright, pruebas API	Pruebas unitarias, integración, E2E y evaluación IA.
CI/CD	GitHub Actions	Lint, typecheck, tests, build, análisis y releases.

Las tecnologías son candidatas hasta que BMAD Architect compruebe compatibilidad, versiones soportadas, licencias, costos, despliegue, límites de proveedor y mantenimiento. Fijar versiones y commits; no consumir "latest" automáticamente.

4.1 Vista lógica

Cliente web/móvil → **API y control de acceso** → **servicios de dominio** → **orquestador de IA** → **RAG / visión / OCR / generación documental** → **MongoDB y almacenamiento de objetos**. Los workers ejecutan procesos largos fuera del ciclo de petición HTTP.

4.2 Separación de responsabilidades

- El backend es la autoridad para permisos, estados, reglas de negocio y autorización.
- El motor IA propone datos y clasificaciones; no escribe directamente estados definitivos sin validación.
- El servicio de archivos valida tipo, tamaño, malware, hash y permisos antes de procesar.
- RAG devuelve fragmentos, metadatos, versión y referencia documental; la respuesta debe citar las fuentes recuperadas.
- La generación PDF utiliza plantillas controladas y datos validados; guardar versión de plantilla y resultado.

5. RAG y gestión del conocimiento

El RAG no debe limitarse a una conversación. Debe permitir respuestas y extracción estructurada basadas en documentos autorizados, mostrando la fuente, sección, versión, fecha de vigencia y fragmentos utilizados.

5.1 Pipeline documental

1. Ingestar documento autorizado y registrar propietario, tipo, jurisdicción, versión, fecha de publicación, vigencia y permisos.
2. Validar formato, integridad y seguridad; ejecutar OCR cuando sea necesario.
3. Normalizar texto conservando páginas, títulos, tablas y referencias.
4. Fragmentar con estrategia adecuada al documento; conservar metadatos y localización.
5. Generar embeddings con versión registrada y almacenar vectores.
6. Recuperar por similitud y filtros; evaluar búsqueda híbrida cuando aporte valor.
7. Construir contexto para el modelo y exigir citas a los fragmentos recuperados.
8. Registrar consulta, documentos recuperados, versión de prompt/modelo y respuesta.

5.2 Reglas del RAG

- No inventar artículos, coberturas, requisitos ni citas.
- Si no hay evidencia suficiente, responder “información insuficiente” y solicitar revisión.
- Aplicar filtros por jurisdicción, producto, tipo documental, vigencia y permisos.
- Distinguir texto normativo, póliza, procedimiento interno y material informativo.
- Resolver conflictos por jerarquía y vigencia mediante reglas documentadas; no por intuición del modelo.
- Versionar documentos y permitir reconstruir qué fuente sustentó cada resultado.

6. Análisis de accidentes y visión artificial

El sistema debe modelar por separado la naturaleza del evento y su posible relación con el trabajo o una póliza personal. Un accidente de tránsito puede ser también potencialmente laboral; no son categorías mutuamente excluyentes.

Dimensión	Ejemplos de valores
Tipo de evento	Tránsito, caída, golpe, corte, quemadura, otro, no determinado.
Contexto laboral	Durante labor, en misión, desplazamiento, trayecto, fuera de jornada, desconocido.
Relación con empleador	Transporte suministrado, actividad ordenada, relación no establecida, por verificar.
Contexto personal	Actividad cotidiana, recreativa, doméstica, viaje, otro.
Estado de evaluación	Pendiente de información, preliminar, revisión humana, confirmado por responsable, no determinado.

6.1 Flujo de análisis

1. Recibir relato y capturar datos estructurados; detectar campos ausentes o contradictorios.
2. Formular preguntas aclaratorias sin sugerir respuestas.
3. Analizar documentos y evidencias visuales por separado.
4. En video, extraer frames con muestreo configurable, límites de duración/resolución y registro de timestamps.
5. Describir solo elementos observables: vehículos, entorno, objetos, secuencia visible y calidad de imagen.

6. Cruzar evidencia con relato y documentación; señalar contradicciones y limitaciones.
7. Generar clasificación preliminar con razones, fuentes, incertidumbre y preguntas pendientes.
8. Enviar a revisión humana cuando exista incertidumbre, contradicción o impacto relevante.

6.2 Límites de visión

- Una fotografía no demuestra por sí sola causa, responsabilidad, fecha ni relación laboral.
- No inferir diagnósticos, gravedad clínica, identidad, intención o culpabilidad.
- No concluir que ocurrió un accidente solo por detectar una lesión o un vehículo.
- Preservar archivo original, hash, metadatos disponibles y cadena de custodia operativa.
- Registrar qué frames fueron analizados y qué modelo/versión produjo cada descripción.

7. Modelo de datos inicial

MongoDB será la base operacional del piloto. Mantener colecciones con esquemas validados, índices, referencias y políticas de acceso. Evitar documentos ilimitados o duplicaciones que impidan auditar cambios.

Colección	Contenido principal
users	Identidad, estado, perfil y referencias de roles.
roles / permissions	Permisos y asignaciones RBAC.
claims	Expediente, propietario, estado, asignación y fechas.
accidents	Hechos reportados, dimensiones del evento y contexto.
people / parties	Personas involucradas y relación con el expediente, minimizando datos.
evidences	Referencia a archivo, hash, MIME, tamaño, timestamps y estado de análisis.
ai_analyses	Entrada, salida estructurada, incertidumbre, modelo, prompt y revisión.
knowledge_documents	Metadatos, versión, vigencia, permisos y estado de indexación.
knowledge_chunks	Fragmentos, referencias, embeddings y metadatos.
forms / form_templates	Plantillas, versiones, campos y reglas de renderizado.
authorizations	Solicitud, actor autorizado, decisión, motivo, fecha y evidencia.
audit_logs	Actor, acción, objeto, timestamp, resultado y correlación.
consents	Finalidad, versión del texto, sujeto, fecha, canal y revocación.

Ejemplo conceptual de expediente

```
{ "claimId": "CLM-000001", "eventType": "traffic_accident", "laborContext": { "status":
"under_evaluation", "employerProvidedTransport": null }, "assessment": { "classification":
"potential_work_related", "status": "preliminary", "humanReviewRequired": true }, "workflowStatus":
"pending_review" }
```

El ejemplo es ilustrativo, no una regla jurídica. No usar un número de confianza como probabilidad legal o garantía de exactitud.

8. Seguridad, privacidad y cumplimiento

El producto puede procesar datos personales y datos sensibles, incluidos datos de salud. Para Colombia, el diseño debe revisarse frente a la Ley 1581 de 2012, sus normas reglamentarias, obligaciones aplicables al responsable/encargado y las reglas sectoriales pertinentes. Se requiere asesoría jurídica antes de operar con casos reales.

- Minimización de datos: recoger solo lo necesario para la finalidad definida.

- Consentimiento y base jurídica: documentar finalidad, información entregada, autorización y excepciones aplicables.
- RBAC y mínimo privilegio; separación por organización/cliente si el producto será multi-tenant.
- Cifrado en tránsito y en reposo; gestión segura de secretos.
- URLs firmadas de corta duración para archivos y control de acceso en cada descarga.
- Auditoría de lectura, modificación, exportación, autorización y eliminación.
- Retención y eliminación documentadas; copias de seguridad y restauración probadas.
- Evaluación de proveedores, transferencias internacionales y tratamiento de datos por API de IA.
- No enviar datos identificables a modelos externos hasta aprobar política, contrato, configuración y base jurídica.
- Plan de respuesta a incidentes y procedimiento de derechos de titulares.

9. Diseño UX/UI

Dirección visual: plataforma empresarial moderna, confiable, accesible y orientada a expedientes. Paleta sugerida: azul profundo, blanco, grises claros y acentos para estados. Tipografía legible, navegación consistente y diseño responsive.

Pantalla	Objetivo
Login y recuperación	Acceso seguro y recuperación de cuenta.
Dashboard	Resumen de expedientes, pendientes, tiempos y alertas.
Bandeja de expedientes	Búsqueda, filtros, asignación y estados.
Nuevo reporte	Wizard conversacional y formulario accesible.
Detalle del expediente	Datos, personas, evidencias, actividad y decisiones.
Análisis IA	Hallazgos, fuentes, incertidumbre y revisión.
Evidencias	Visor de imágenes/video/documentos y metadatos.
Formularios	Previsualización, corrección, aprobación y PDF.
Autorizaciones	Bandeja de aprobación y registro de decisión.
Base de conocimiento	Carga, versiones, vigencia, indexación y pruebas RAG.
Usuarios y roles	Administración de identidad y permisos.
Auditoría	Historial filtrable y exportación controlada.

10. BMAD Method: gobierno del desarrollo

BMAD será el marco de análisis, planificación, arquitectura, UX, desarrollo asistido y verificación. La versión y módulos exactos se deben confirmar al instalar; no asumir que un agente o workflow existe en todas las distribuciones.

Rol/agente	Responsabilidad	Artefactos
Analyst	Investigar problema, usuarios, procesos y riesgos.	Product Brief, research notes.
PM	Definir alcance, requisitos, prioridades e historias.	PRD, roadmap, backlog.
Architect	Diseñar límites, stack, contratos, datos y decisiones.	Architecture, ADR, API/data contracts.
UX Designer	Flujos, wireframes, componentes y accesibilidad.	UX specification, design system.
Developer	Implementar historias aprobadas y pruebas.	Código, tests, PR.
QA / Test Architect	Estrategia, cobertura, casos y evaluación.	QA strategy, test plans, defect reports.
Release Manager	Preparar notas, tags y checklist de release.	Changelog, release notes.

10.1 Autonomía por niveles

Nivel	Acciones permitidas	Control
1 — Autónomo	Documentación, componentes locales, refactor seguro, pruebas unitarias.	Pruebas y lint automáticos.
2 — Autónomo con gates	Elegir componentes de plantillas aprobadas, librerías permitidas, optimizaciones no disruptivas.	Build, tests, revisión de dependencias y políticas.
3 — Revisión arquitectónica	Cambiar base de datos, proveedor IA, contratos API, permisos o dependencias críticas.	ADR y revisión del responsable del producto/arquitectura.
4 — Aprobación humana obligatoria	Producción, borrado de datos, autorizaciones definitivas, decisiones de cobertura, cambios legales o tratamiento sensible.	Aprobación explícita y registro auditable.

Los agentes pueden resolver tareas dentro de límites aprobados, pero no deben autoaprobar su propio trabajo crítico. El agente que implementa no es el único que valida.

10.2 Qué significa preparar o “entrenar” agentes

- Instrucciones estables por rol: misión, alcance, entradas, salidas y restricciones.
- Contexto de proyecto: Product Brief, PRD, arquitectura, glosario y decisiones.
- Conocimiento recuperable: documentación de plantillas, estándares, ejemplos y patrones aprobados.
- Herramientas permitidas: lectura/escritura de repositorio, terminal, tests, GitHub y documentación según permisos.
- Workflows repetibles: análisis de historia, selección de plantilla, implementación, QA y release.
- Evaluaciones: tareas de referencia, resultados esperados, métricas y regresiones.
- Versionado de prompts, instrucciones, workflows y configuración de modelos.

No incluir secretos, tokens, datos reales de siniestros ni documentos confidenciales en el contexto general de los agentes. Usar datos sintéticos para desarrollo y pruebas.

11. Template Registry: biblioteca de plantillas

Las plantillas se obtendrán de repositorios oficiales o mantenedores reconocidos. Cada una debe tener ficha, URL de origen, licencia, versión/tag, commit, fecha de evaluación, dependencias, instrucciones de uso, vulnerabilidades conocidas y estado de aprobación.

Categoría	Candidatos a evaluar	Uso previsto
Web React	Next.js starter, shadcn/ui, TailAdmin, Refine	Web pública/privada, dashboard, componentes.
Backend Node	NestJS TypeScript starter, Mongoose patterns	API modular y persistencia MongoDB.
Móvil	Expo + React Native starter	Aplicación nativa Android/iOS.
Monorepo	Turborepo + pnpm workspaces	Apps y paquetes compartidos.
QA	Playwright, Vitest/Jest, ESLint, TypeScript	E2E, unit tests, lint y tipos.
IA/RAG	Patrones oficiales de proveedor y ejemplos mantenidos	Ingesta, recuperación, evaluación y visión.

Fuentes oficiales para comenzar (validar licencia y estado antes de adoptar):

- Next.js: https://nextjs.org/docs · https://github.com/vercel/next.js
- shadcn/ui: https://ui.shadcn.com/ · https://github.com/shadcn-ui/ui
- TailAdmin: https://tailadmin.com/ · https://github.com/TailAdmin/free-react-tailwind-admin-dashboard
- Refine: https://refine.dev/ · https://github.com/refinedev/refine
- NestJS: https://nestjs.com/ · https://github.com/nestjs/typescript-starter
- Mongoose: https://mongoosejs.com/
- Expo: https://docs.expo.dev/ · https://github.com/expo/expo
- Turborepo: https://turbo.build/repo/docs
- BMAD: https://github.com/bmad-code-org/BMAD-METHOD

No copiar plantillas completas sin revisar licencia, atribuciones, dependencias, telemetría, autenticación preconfigurada y vulnerabilidades. Preferir instalar dependencias o usar starters limpios, y conservar los avisos de licencia.

11.1 Ficha mínima de plantilla

```
template: id: frontend-nextjs-enterprise category: frontend source_url: https://github.com/... version:
exact-tag commit: exact-commit-sha license: verified-license reviewed_at: YYYY-MM-DD compatible_with:
node: supported-version typescript: supported-version checks: build: passed tests: passed
security_review: passed status: approved | rejected | pending
```

12. Repositorios y estructura de trabajo

Separar el entorno de ingeniería reutilizable del producto comercial:

Repositorio	Propósito
INSURE-AI-DEVELOPMENT-ENVIRONMENT	BMAD, módulos propios, agentes, workflows, políticas y Template Registry.
INSURE-AI-PLATFORM	Código del producto, documentación específica, pruebas, infraestructura y releases.

Estructura recomendada del repositorio del producto (monorepo):

```
insure-ai-platform/ apps/ web/ # Next.js mobile/ # Expo / React Native api/ # NestJS workers/ # trabajos
asíncronos packages/ contracts/ # contratos y tipos validation/ # validaciones compartidas api-client/ #
SDK de API domain/ # reglas de dominio puras ui-web/ # componentes web docs/ BMAD/ architecture/ UX/ API/
SECURITY/ RELEASES/ infrastructure/ tests/ integration/ e2e/ security/ ai-evaluation/ policies/
.github/workflows/ pnpm-workspace.yaml turbo.json README.md CHANGELOG.md
```

No compartir componentes web con React Native salvo que la librería sea compatible. Compartir preferentemente contratos, validaciones, lógica pura y cliente API.

13. GitHub, ramas y versionamiento

GitHub será la fuente oficial de código, issues, decisiones y releases. Proteger main, requerir Pull Request, checks automáticos y revisión. No subir .env, claves, tokens, archivos reales de clientes ni videos/fotos sensibles.

Rama	Uso
main	Código estable y releases.
develop	Integración continua, si el flujo del equipo la necesita.
feature/-nombre	Nueva funcionalidad vinculada a historia.
fix/-nombre	Corrección de defecto.
release/x.y.z	Preparación y estabilización de release.
hotfix/x.y.z	Corrección urgente sobre versión liberada.

13.1 Roadmap inicial de versiones

Versión	Hito	Criterio de salida
v0.1.0	Foundation	Repositorios, políticas, CI, estructura y documentación inicial.
v0.2.0	UI/UX	Design system, login visual, navegación y dashboard inicial.
v0.3.0	Core backend	Usuarios, roles, MongoDB, auditoría y contratos API.
v0.4.0	Claims	Registro y gestión de expedientes y accidentes.
v0.5.0	AI Engine	Extracción estructurada y clasificación preliminar evaluada.
v0.6.0	RAG	Ingesta, indexación, recuperación y citas.
v0.7.0	Computer Vision	Carga, frames, análisis visual y asociación de evidencias.
v0.8.0	Workflow	Formularios, autorizaciones y revisión humana.
v0.9.0	QA	Pruebas integrales, seguridad y correcciones.
v1.0.0	Pilot Release	Piloto controlado, documentación, métricas y aprobación.

Versionar también prompts, esquemas de salida, configuraciones de modelos, pipeline de embeddings, corpus documental e índices. La versión del software no reemplaza la versión del modelo ni del conocimiento.

14. Plan de ejecución

Estimación inicial de diez semanas, sujeta a ajuste tras el análisis BMAD, disponibilidad de documentación y complejidad de integraciones.

Semana	Trabajo	Entregable
1	Analysis	Product Brief, mapa de usuarios/procesos y riesgos.
2	Planning	PRD, requisitos, historias y criterios de aceptación.
3	Solutioning + UX	Arquitectura, ADR, contratos, diseño UX y QA strategy.
4	Sprint 0	BMAD, plantillas, políticas, monorepo y CI.
5–6	Core	Identidad, RBAC, expedientes, datos y formularios base.
7	IA	Adaptador de modelo, extracción estructurada y evaluación inicial.
8	RAG + visión	Ingesta, búsqueda, imágenes y frames de video.

Semana	Trabajo	Entregable
9	Workflow	Autorizaciones, revisión humana, PDF y auditoría.
10	QA + piloto	Pruebas, seguridad, aceptación y release candidata.

15. Sprint 0 — habilitar el entorno autónomo

Duración sugerida: cinco días laborables. Objetivo: demostrar que el entorno puede producir una funcionalidad pequeña de extremo a extremo bajo políticas y controles.

Día	Actividad	Evidencia de terminación
1	Preparar repositorio de entorno, instalar BMAD y confirmar módulos/workflows disponibles.	Instalación reproducible y versión documentada.
2	Evaluar y registrar starters React, Node, Expo y monorepo.	Template Registry con licencia, tag/commit y checks.
3	Crear políticas de autonomía, seguridad, coding standards y arquitectura.	Políticas versionadas y revisadas.
4	Crear monorepo del producto y pipeline CI.	Lint, typecheck, tests y build ejecutables.
5	Ejecutar prueba de autonomía con historia pequeña.	PR de muestra, informe QA y retrospectiva.

Prueba recomendada: historia de gestión básica de usuarios o catálogo no sensible. Debe incluir contrato API, UI, persistencia, permisos, pruebas y documentación. No comenzar con clasificación automática de siniestros como primera prueba.

16. Quality Gates y criterios de aceptación

- Lint, formato, typecheck y build pasan en CI.
- Pruebas unitarias y de integración ejecutadas; cobertura definida por módulo, no un porcentaje arbitrario global.
- Pruebas E2E para flujos críticos de usuario.
- Autorización RBAC probada en API, no solo ocultando botones en frontend.
- Validación de entrada, manejo de errores, límites de carga y protección de archivos.
- Escaneo de secretos, dependencias y vulnerabilidades.
- Pruebas de aislamiento entre expedientes/organizaciones cuando aplique.
- RAG evaluado con conjunto de preguntas y respuestas de referencia; medir recuperación, exactitud de citas y abstención.
- Clasificación IA evaluada con dataset etiquetado y revisión experta; medir falsos positivos/negativos por categoría.
- Toda respuesta generada debe guardar trazabilidad de modelo, prompt, fuentes y revisión.
- No release estable con defectos críticos o vulnerabilidades críticas sin mitigación aprobada.

17. Métricas del piloto

Área	Métricas sugeridas
Operación	Tiempo de creación, tiempo de procesamiento, expedientes pendientes, fallos de carga.
Extracción	Exactitud por campo, campos omitidos, tasa de corrección humana.
Clasificación	Matriz de confusión, precisión/recall por clase, abstención y desacuerdo experto.
RAG	Recall@k, relevancia, exactitud de citas, respuestas sin soporte y tasa de abstención.

Área	Métricas sugeridas
Visión	Porcentaje de archivos procesables, frames útiles, errores y limitaciones detectadas.
Seguridad	Accesos indebidos bloqueados, eventos auditados, incidentes y hallazgos.
Experiencia	Tasa de finalización, abandono, tiempo por formulario y satisfacción.

18. Decisiones pendientes que BMAD debe resolver

No asumir respuestas. El Analyst/PM debe levantar estas preguntas y el Architect documentar la recomendación y sus consecuencias.

1. ¿El piloto operará únicamente en Colombia? ¿Qué jurisdicciones y marcos sectoriales aplican?
2. ¿La plataforma será para una sola organización o multi-tenant desde el inicio?
3. ¿Qué tipos de seguro, pólizas y procesos se atenderán en la primera versión?
4. ¿Quiénes son los usuarios reales y qué roles pueden reportar, revisar, aprobar o administrar?
5. ¿Qué formularios oficiales o internos se deben reproducir? ¿Quién es propietario de cada plantilla?
6. ¿Qué documentos normativos y contractuales están autorizados para el corpus RAG?
7. ¿Qué proveedor y modelo se utilizará para texto, visión y embeddings? ¿Qué condiciones de privacidad y retención ofrece?
8. ¿Qué volumen estimado de expedientes, fotos, minutos de video y usuarios concurrentes se espera?
9. ¿Dónde se desplegará y cuál es el presupuesto mensual/anual real?
10. ¿Qué nivel de disponibilidad, respaldo, recuperación y retención se requiere?
11. ¿La app móvil necesita operación offline y sincronización posterior?
12. ¿Qué firma electrónica, consentimiento y evidencia de autorización se requieren?

19. Decisiones arquitectónicas que deben registrarse como ADR

- ADR-001: monorepo y estrategia de aplicaciones.
- ADR-002: Next.js/PWA y estrategia Expo/React Native.
- ADR-003: NestJS y modularización del backend.
- ADR-004: MongoDB, Mongoose e índices.
- ADR-005: almacenamiento de objetos y ciclo de vida de evidencias.
- ADR-006: proveedor IA y adaptador de modelos.
- ADR-007: motor RAG, embeddings y estrategia de búsqueda.
- ADR-008: procesamiento de video y límites de frames.
- ADR-009: autenticación, RBAC y multi-tenancy.
- ADR-010: despliegue, CI/CD, observabilidad y costos.

20. Instrucción inicial para Claude Code + BMAD

Copiar el siguiente texto como primera solicitud después de abrir el repositorio del entorno. Claude debe leer el PDF y no comenzar a implementar el producto hasta completar el análisis.

```
Actúa como coordinador técnico del proyecto INSURE AI. Este documento PDF es el contexto inicial y no sustituye los artefactos oficiales de BMAD. OBJETIVO: Preparar un entorno de ingeniería asistida por agentes, con BMAD Method, para diseñar y desarrollar una plataforma multiplataforma de gestión de accidentes, expedientes, evidencias, formularios y autorizaciones, con IA generativa, RAG y análisis visual. REGLAS: 1. Inspecciona el repositorio, la versión instalada de BMAD y los módulos/workflows realmente disponibles. 2. No inventes agentes, comandos, módulos ni capacidades. Verifica la documentación instalada. 3. Ejecuta primero Analysis y Planning. No generes código de producto todavía. 4. Identifica requisitos ambiguos, riesgos, dependencias, decisiones pendientes y supuestos. 5. Produce Product Brief, PRD, glosario, mapa de actores, procesos, historias y criterios de aceptación. 6. En Solutioning, compara tecnologías y plantillas con versiones, licencias, mantenimiento, seguridad y costo. 7. Crea ADR para decisiones arquitectónicas y solicita revisión humana para decisiones de nivel 3 y 4. 8. Construye Template Registry con URL, licencia, tag, commit, fecha de revisión, dependencias y resultados de pruebas. 9. Configura agentes y workflows con instrucciones, permisos mínimos, contexto y quality gates. 10. Nunca uses datos reales sensibles en ejemplos, prompts de prueba o repositorios. 11. La IA no toma decisiones definitivas de cobertura, responsabilidad, diagnóstico ni autorización. 12. Todo
```

```
cambio debe quedar trazado en GitHub mediante historias, commits, Pull Requests y releases. PRIMER ENTREGABLE: Un informe de diagnóstico con: - artefactos BMAD disponibles y faltantes; - plan de instalación/configuración; - preguntas abiertas priorizadas; - riesgos; - propuesta de estructura de repositorios; - plan detallado de Sprint 0; - criterios objetivos para demostrar autonomía. Espera la aprobación del Product Owner antes de iniciar la implementación del producto.
```

21. Instrucción para agentes individuales

Cada agente debe recibir un contrato similar, adaptado a su rol:

```
ROL: [NOMBRE DEL AGENTE] MISIÓN: [RESULTADO QUE DEBE PRODUCIR] CONTEXTO OBLIGATORIO: PRD, arquitectura, ADR, políticas y story asignada. ENTRADAS: historia, criterios de aceptación, contratos y plantilla aprobada. PERMISOS: archivos, comandos y herramientas autorizadas. RESTRICCIONES: no cambiar stack/contratos sin ADR; no exponer secretos; no usar datos reales; no tomar decisiones reservadas a humanos. SALIDAS: artefactos, cambios, pruebas, evidencias y riesgos. DEFINITION OF DONE: - criterios de aceptación cumplidos; - lint/typecheck/build correctos; - pruebas ejecutadas y resultados registrados; - documentación actualizada; - riesgos y limitaciones declarados; - PR listo para revisión. Si falta información crítica, detenerse y registrar la pregunta; no completar el vacío mediante una suposición silenciosa.
```

22. Glosario

Término	Definición en este proyecto
Claim / expediente	Registro integral de un evento reportado y su gestión.
RAG	Generación aumentada por recuperación de documentos relevantes.
Embedding	Representación vectorial utilizada para búsqueda semántica.
Frame	Fotograma extraído de un archivo de video.
RBAC	Control de acceso basado en roles.
ADR	Architecture Decision Record: registro de una decisión técnica.
PWA	Aplicación web progresiva instalable y adaptable.
Quality Gate	Condición verificable que debe superarse para avanzar.
Agente	Rol de IA configurado con instrucciones, contexto y herramientas.
Human-in-the-loop	Revisión o decisión humana dentro del proceso automatizado.

23. Referencias técnicas oficiales

- BMAD Method: https://github.com/bmad-code-org/BMAD-METHOD
- Next.js: https://nextjs.org/docs
- shadcn/ui: https://ui.shadcn.com/
- NestJS: https://docs.nestjs.com/
- MongoDB: https://www.mongodb.com/docs/
- Mongoose: https://mongoosejs.com/docs/
- Expo: https://docs.expo.dev/
- React Native: https://reactnative.dev/docs/getting-started
- Turborepo: https://turbo.build/repo/docs
- Playwright: https://playwright.dev/docs/intro
- Vitest: https://vitest.dev/guide/
- FFmpeg: https://ffmpeg.org/documentation.html
- Anthropic API: https://docs.anthropic.com/

- Colombia — Ley 1581 de 2012: consultar texto oficial en SUIN-Juriscol y autoridad competente.

24. Definición de éxito

INSURE AI estará listo para el piloto cuando un usuario autorizado pueda reportar un accidente, completar información faltante, aportar evidencias, obtener un análisis preliminar sustentado en fuentes autorizadas, revisar y corregir los datos extraídos, generar el formulario correspondiente y seguir el expediente mediante un flujo auditable; todo ello con controles de acceso, protección de datos, métricas de calidad y revisión humana para decisiones críticas.

Fin del documento maestro · INSURE AI v1.0